

WEEK 2 · DAY 2

IAM Policies

— Writing & Reading Permissions

BaseStack Academy · AWS Cloud Accelerator

IAM & Security



Today's Agenda

IAM Policies — Writing & Reading Permissions

1 Overview — Why This Matters

FOUNDATION

2 Key Concepts — Definitions & Vocabulary

CORE KNOWLEDGE

3 Deep Dive — Key Concepts Extended

DEEPER LEARNING

4 How It Works — Step by Step

MECHANICS

5 Technical Deep Dive

ADVANCED DETAIL

6 Architecture & Visual Model

VISUAL

7 Real-World Nigerian Scenario

CONTEXT

8 Common Mistakes & Exam Traps

EXAM PREP

Overview — Why This Matters

IAM Policies are the mechanism through which permissions are granted in AWS. Every action you take in AWS — creating an EC2 instance, uploading a file to S3, calling a Lambda function — is checked against IAM policies. If no policy explicitly allows an action, it is denied by default. This “default deny” model means AWS is secure out of the box, and access must be intentionally granted.

Understanding policy structure is essential for both the SAA-C03 exam and real-world work. You do not need to write complex JSON from scratch — AWS provides hundreds of managed policies. But you need to read policies, understand what they permit, and know when to use managed policies versus creating your own custom policies.

WHY NOW?

Cloud skills are the #1 most in-demand tech capability globally. Understanding this topic is essential for both the SAA-C03 exam and real cloud engineering work.



Beginner Note

Do not worry if this feels new — every expert was once a complete beginner. Take notes and ask questions freely.



Key Concepts — Know These Definitions

📖 Study tip: Say each definition aloud — spoken repetition builds memory 3× faster than reading alone.

Managed Policies

Pre-built policies created and maintained by AWS (e.g., AmazonS3ReadOnlyAccess) or by you (customer managed)

Inline Policies

Policies embedded directly inside a single user, group, or role — not reusable

Policy Structure

Effect (Allow/Deny), Action (which AWS API calls), Resource (which specific resources), Condition (optional restrictions)

Explicit Deny

An explicit Deny in any policy ALWAYS overrides any Allow — even from a managed policy

Policy Simulator

AWS IAM Policy Simulator lets you test policies before applying them — no real-world impact

Key Concepts (Continued)

Concepts Summary

All key concepts for this lesson have been covered. Use this slide to review and quiz yourself.

✓ Managed Policies

✓ Inline Policies

✓ Policy Structure

✓ Explicit Deny

✓ Policy Simulator

⚙️ How It Works — Step by Step

1

Every IAM policy is a JSON document with a specific structure. The core elements are:

2

Effect: Allow or Deny — determines what the policy does. Action: The specific AWS API call, e.g., `s3:GetObject` to read files, `ec2:StartInstances` to start servers, or `s3:*` for all S3 actions. Resource: The ARN of the target — e.g., `arn:aws:s3:::my-bucket/*` or use `*` for all resources. Condition: Optional constraints — restrict by IP address, require MFA, or limit to specific AWS regions.

3

AWS evaluates permissions by collecting ALL policies attached to the user (directly or via groups), plus any resource-based policies, plus any Service Control Policies (SCPs) from AWS Organizations. Rule 1: If ANY policy has an explicit Deny, the request is denied — no Allow can override it. Rule 2: If no policy has a matching Allow, the request is also denied. This is the default-deny model: in AWS, every action is blocked unless explicitly permitted by at least one policy.

Reading a Real IAM Policy — JSON Breakdown

Example: Customer-Managed Policy (Emeka's S3 Access)

```
{ "Version": "2012-10-17",  
  "Statement": [{ "Effect": "Allow",  
    "Action": ["s3:GetObject", "s3:ListBucket"],  
    "Resource": ["arn:aws:s3:::company-app-bucket",  
      "arn:aws:s3:::company-app-bucket/*"] } ] }
```

What this policy does:

- ✓ Read and list files in company-app-bucket only
- ✗ Cannot delete, upload, or access billing or any other bucket
- ✗ Cannot access EC2, RDS, Lambda, or any other AWS service

Note: Two ARNs are needed — one for the bucket itself and one for its objects (/*). Missing either breaks the policy.

? Architecture & Visual Model

Core Components

Managed Policies

Pre-built policies created and maintained by AWS (for example, AmazonS3ReadOnlyAccess) or created and managed by you.

Inline Policies

Policies embedded directly inside a single user, group, or role — not reusable

Policy Structure

Effect (Allow or Deny), Action (specific AWS API calls), Resource (targeted resources), Condition (rules that control when access applies).

How They Connect

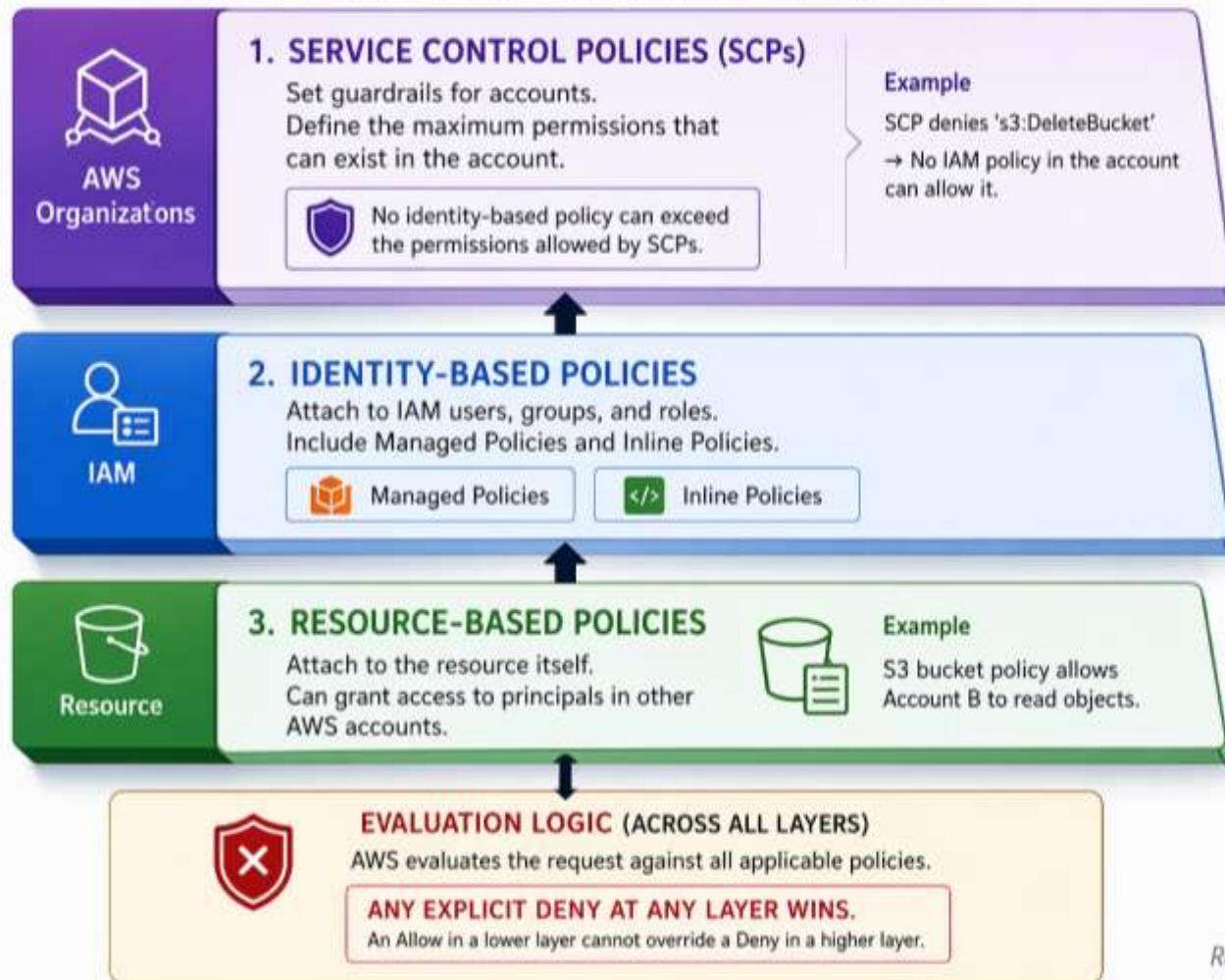
Explicit Deny

An explicit Deny in any policy ALWAYS overrides any Allow — even from a managed policy

Policy Simulator

AWS IAM Policy Simulator lets you test policies before applying them — no real-world impact

AWS POLICY LAYERS (EVALUATED TOGETHER)



KEY TAKEAWAYS



SCPs are the top guardrails. They set the maximum permissions for an account.



Identity-based policies define what identities (users, groups, roles) are allowed to do.



Resource-based policies are attached to resources and can grant access across AWS accounts.



Explicit Deny anywhere overrides everything.



AWS evaluates all layers together to make the final decision.

Remember: Least privilege + Guardrails + Monitoring



Real-World Nigerian Scenario

Emeka is a junior developer at a Lagos startup. His manager creates a customer managed policy allowing `s3:GetObject` and `s3:ListBucket` on the company's application bucket only — not all S3 buckets. Emeka can read application files but cannot read billing reports stored in a different bucket. He cannot delete files, upload new ones, or access any other AWS service. When Emeka accidentally tries to launch an EC2 instance, it is immediately denied — not because anyone saw him try, but because no IAM policy attached to his user grants `ec2:RunInstances`. The permission check is automatic, instant, and requires no human review. AWS denies the request at the API level before anything is provisioned. This is the default-deny model working exactly as designed.

Discussion Question

Emeka's manager needs to give him temporary upload access during a deployment window tonight. What specific IAM change would you make — and why is it critical to revert or time-restrict it immediately after the deployment completes?

⚠️ Common Mistakes & Exam Traps

- These are the most common exam errors — every one costs marks. Memorise them.



Explicit DENY always wins. If a policy allows S3 access but another policy (perhaps an SCP) explicitly denies it, access is denied — period. No Allow can override an explicit Deny.



Using '*' for Resource means ALL resources of that type. S3:DeleteBucket with Resource: '*' means the user can delete ANY bucket. Always scope resources to the minimum needed.



The principle of least privilege is not just good practice — it is an exam concept. When asked which permissions to grant, the answer always involves the minimum required for the specific task.



Question

A developer at a Nigerian fintech company has an IAM identity-based policy granting s3:PutObject on Resource: "*" (all S3 buckets). The company's AWS Organizations SCP explicitly denies s3:PutObject on the production bucket. What happens when the developer tries to upload a file to the production bucket?

A

Access is denied. An SCP explicit Deny overrides all IAM identity-based Allow policies — no user-level policy can grant access that an SCP has blocked.

✓ CORRECT

B

Access is granted. IAM identity-based policies are evaluated before SCPs, so the Allow takes effect first.

C

Access is granted. Using Resource: "*" in an IAM policy grants unrestricted access that cannot be blocked by other policies.

D

Access is denied only if the production bucket also has a resource-based policy that explicitly denies the action.



Question

A developer at a Nigerian fintech company has an IAM identity-based policy granting s3:PutObject on Resource: "*" (all S3 buckets). The company's AWS Organizations SCP explicitly denies s3:PutObject on the production bucket. What happens when the developer tries to upload a file to the production bucket?

A

Access is denied. An SCP explicit Deny overrides all IAM identity-based Allow policies — no user-level policy can grant access that an SCP has blocked.

✓ CORRECT

B

Access is granted. IAM identity-based policies are evaluated before SCPs, so the Allow takes effect first.

C

Access is granted. Using Resource: "*" in an IAM policy grants unrestricted access that cannot be blocked by other policies.

D

Access is denied only if the production bucket also has a resource-based policy that explicitly denies the action.

💡 Key Takeaways — What You Learned Today

IAM Policies — Writing & Reading Permissions

✓ Concepts Covered

✓ Managed Policies

✓ Inline Policies

✓ Policy Structure

✓ Explicit Deny

✓ Policy Simulator

📁 Before Next Class

1. Review the concepts above without notes

2. Complete the hands-on console lab

3. Post your progress on LinkedIn / X

4. Review the exam traps from the previous slide

5. Check the Further Resources list



Further Resources & Reading

Go deeper with these official AWS resources:

1. IAM Policies User Guide

docs.aws.amazon.com/IAM/latest/UserGuide/access_policies.html

2. IAM JSON Policy Reference (all keywords & conditions)

docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies.html

3. IAM Policy Simulator (test policies before applying them)

policysim.aws.amazon.com

4. AWS Security Best Practices (IAM chapter)

docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

5. AWS Policy Generator (build policies visually)

awspolicygen.s3.amazonaws.com/policygen.html



Homework: Read at least ONE link above before the next class. Post what you learned on the WhatsApp group.



BASE STACK

A C A D E M Y

Week 2 · Day 2 Complete

IAM Policies — Writing & Reading Permissions

IAM & Security

Next: Hands-on Console Lab →